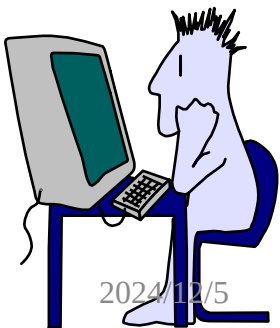


Java网络程序设计

高祥



2024/12/5

Xiang Gao



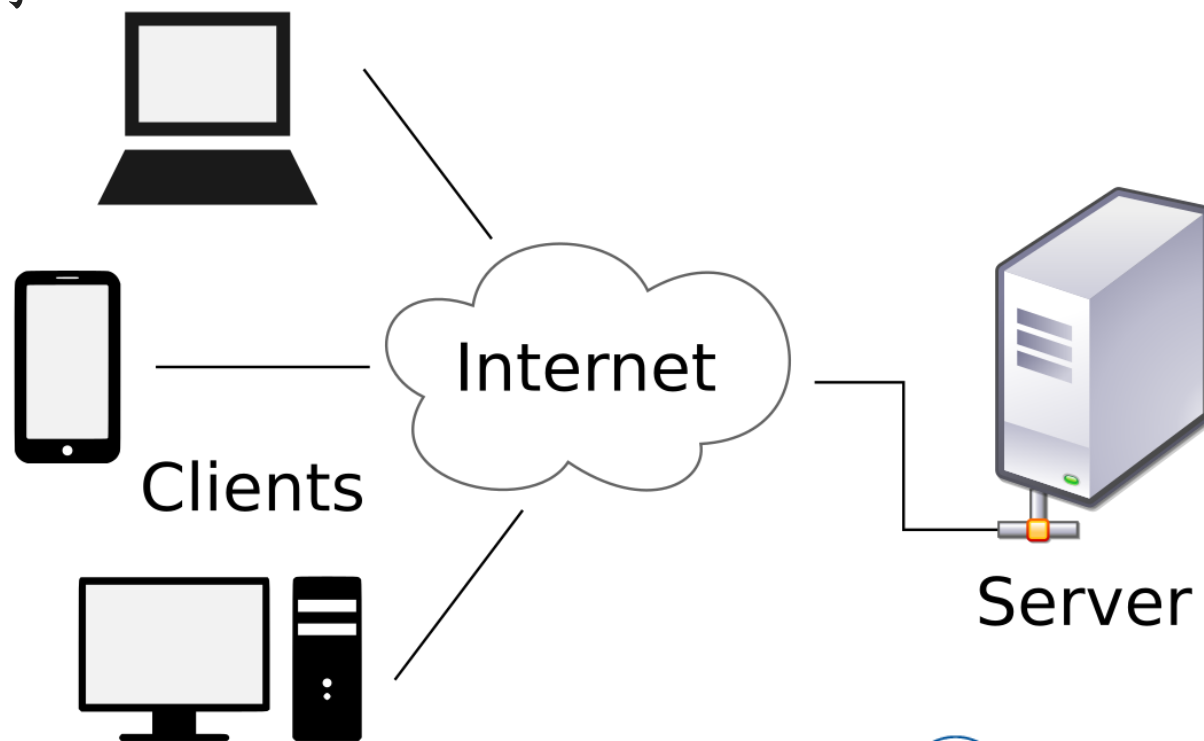
北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院

- 网络编程的基础知识
- 基于TCP的网络程序设计
- 基于UDP的网络程序设计
- 访问Internet网络资源

计算机网络工作模式

- 客户机/服务器模式(Client/Server. C/S)

- C/S 架构是一种典型的两层架构，即客户端服务器端架构，其客户端包含一个或多个在用户的电脑上运行的程序，而服务器端为客户端访问提供支持



Xiang Gao

C/S 架构

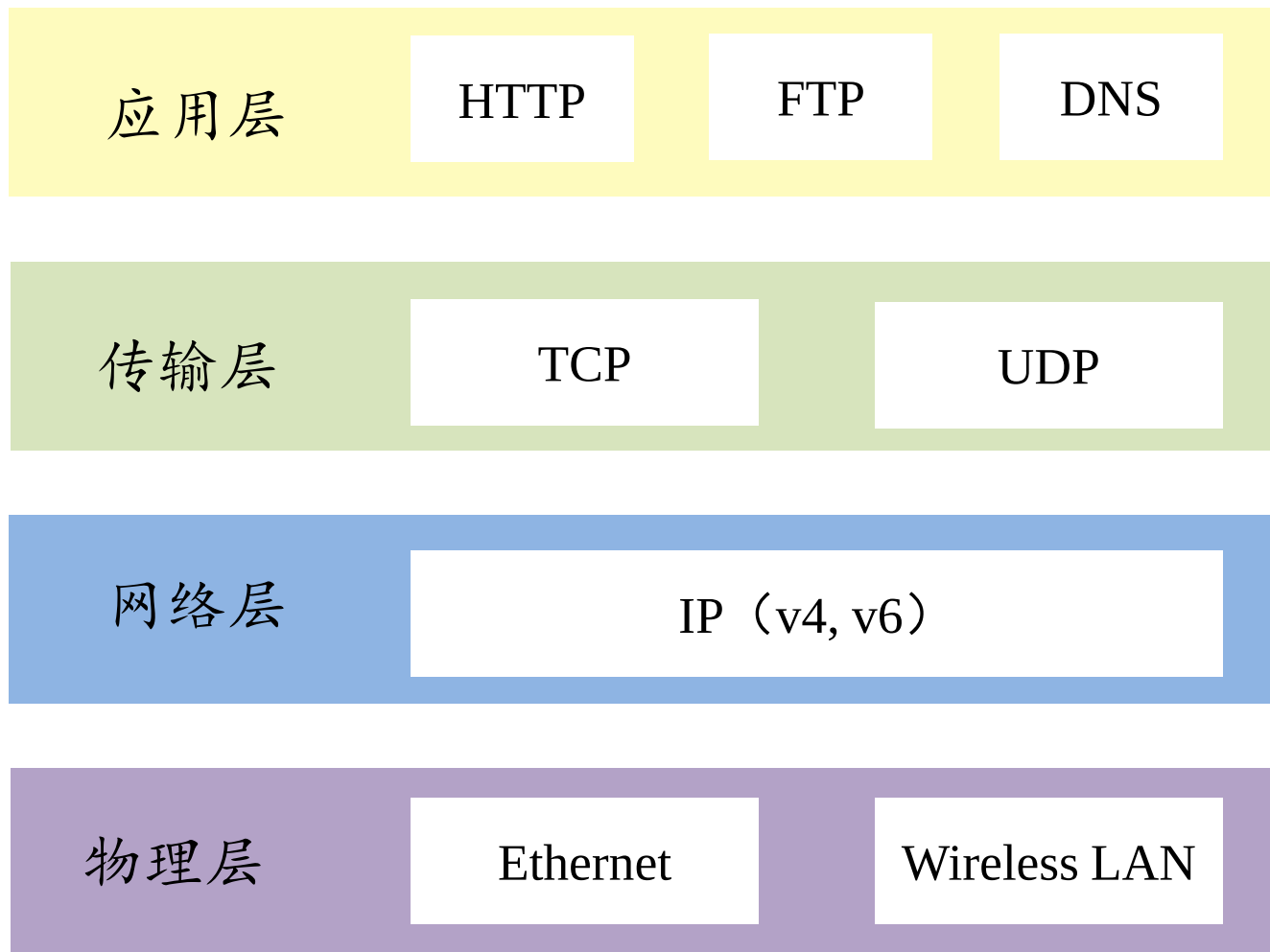
- C/S 架构中客户端需要实现绝大多数的业务逻辑和界面展示：
- 优点：
 - C/S架构的界面和操作可以很丰富。
 - 安全性能比较容易保证，实现多层认证也不难。
 - 由于只有一层交互，因此响应速度较快。
- 缺点：
 - 用户群固定：由于程序需要安装才可使用，因此不适合面向一些不可知的用户。
 - 维护成本高：发生一次升级，则所有客户端的程序都需要改变。
- Peer to Peer Network
- Browser/Server Network

网络编程的基础知识 (1)

- 通信与协议

- 网络进程之间通信必须遵循预定的规则，这些规则叫做网络进程通信协议（传输代码、内容结构、传输控制步骤、出错控制、加密算法等）
- TCP/IP是一组在Internet网络上的不同计算机之间进行通信的协议的总称；从下往上可视为4层结构：物理层、网络层、传输层和应用层。
- TCP/IP由应用层的HTTP、FTP、SMTP和传输层的TCP（传输控制协议）、UDP（用户数据报协议）以及网络层的IP（Internet协议）等一系列协议组成。

网络编程的基础知识(1) TCP/IP架构



网络编程的基础知识 (2)

- 数据的封装与拆封

- **封装**: 发送方数据在网络模型的各层传送过程中加入头尾的过程;
- **拆封**: 接受方收到数据后去除相应的头尾的过程。

TCP、UDP的数据帧格式简单图例:

协议类型	源IP	目标IP	源端口	目标端口	帧序号	帧数据
------	-----	------	-----	------	-----	-----

其中协议类型用于区分TCP, UDP

网络编程的基础知识 (3)

- 在网络上，计算机通过**IP地址或主机名**进行标识，这样使得位于不同地理位置的计算机有可能互相访问和通信。
- 每台计算机都可以存放一定数量的**资源**，并通过网络共享。
 - **网络资源定位器 (URL, Uniform Resource Locator)** 可以指向网络上的各种资源。通过网络资源定位器可以获取网络上的资源。

网络编程的基础知识 (3)

- 网络资源不仅可以包括网络上各种简单对象：
 - 例如网络上的路径和文件(Web页、文本文件、图形(像)文件、声音片断)等；
- 还可以是一些复杂的对象：
 - 如数据库或搜索引擎。
- URL: 是WWW资源统一资源定位器的缩写。他规范了WWW资源网络定位地址的表示方法。

网络编程的基础知识 (4)

- IP地址和Port(端口号)
 - ip是由32位的数字表示的,通常被分成四部分。例如:192.168.0.1
 - 本地回路的IP地址: 127.0.0.1 或localhost
 - Port(端口号):逻辑意义上的数据传输通道,或者说模拟通道。
 - 计算机与网络一般只有一个物理连接,网络数据通过这个连接流向计算机。但是计算机上一般有许多进程在同时运行,为确定网络数据流向的进程,TCP/IP协议引入了端口的概念。
 - 端口同一台网络计算机的一个特定进程关联,与进程建立的套接口绑定在一起。
 - 端口号的范围为0~65535之间,0~1023之间的端口数是用一些知名的网络服务和应用(我们一般不使用)
 - 例如:Web服务使用端口80,FTP服务使用端口21等
 - 当一个互联网上的服务使用一个非默认的ip和port号码时,是这样表示的192.168.0.2:8080

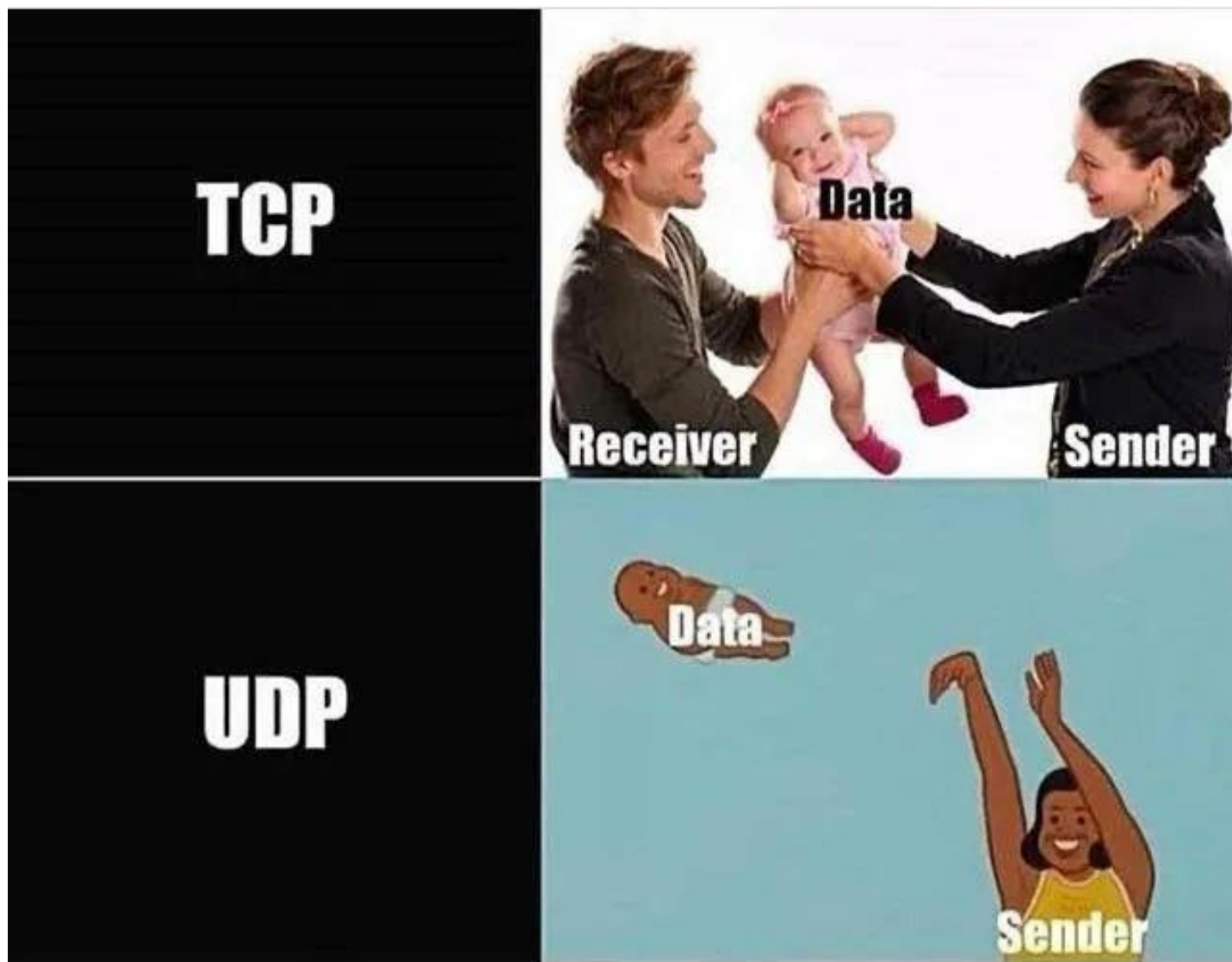
URL (Uniform Resource Locator)

- 网络资源定位器通常有5个部分组成：
 - 协议、主机名、端口号、文件和引用。
- URL的基本表示格式是：
 - *Protocol://hostname:/resourcenam#anchor*
 - Protocol, 使用的协议, 它可以是http、ftp、news、telnet等。
 - Hostname: 主机名, 指定域名服务器(DNS)能访问到的WWW服务的计算机, 例[www. sun. com](http://www.sun.com)
 - Port: 是可选的, 表示所连接的端口号, 如缺省, 将连接到协议缺省的端口。
 - Resourcenam: 资源名, 是主机上能访问到的目录或文件。
 - Anchor: 标记, 是可选的。他指定文件中有特定标记的位置。

URL (Uniform Resource Locator)

- 例如:
- <http://www.sina.com.cn:8080/index.html>
- `http://www.gamelan.com:80/Gamelan/network.html#BOTTOM`
- `http://www.buaa.edu.cn/library/library.html`

网络编程的基础知识TCP/UDP (5)



网络编程的基础知识TCP/UDP (5)

- UDP与TCP
 - TCP, 传输控制协议(Transmission Control Protocol), 是面向连接的通信协议。
 - 使用TCP协议进行数据传输时,两个进程之间会建立一个连接,数据以流的形式顺序传输。
 - UDP, 用户数据协议(User Datagram Protocol), 是无连接通信协议。
 - 使用UDP协议进行数据传输时,两个进程之间不建立特定的连接,不对数据到达的顺序进行检查。
- 在互联网上进行数据传输,多用TCP和UDP协议,它们传输的都是一个byte stream/字节型的数据流

网络编程的基础知识TCP/UDP (5)

	TCP	UDP
可靠性	可靠	不可靠
连接性	面向连接	无连接
报文	面向字节流	面向报文
效率	传输效率低	传输效率高
双工性	全双工	一对一、一对多、多对一、多对多
流量控制	滑动窗口	无
拥塞控制	慢开始、拥塞避免、快重传、快恢复	无
传输速度	慢	快
应用场景	对效率要求低，对准确性要求高或者要求有连接的场景	对效率要求高，对准确性要求低

什么是网络编程？

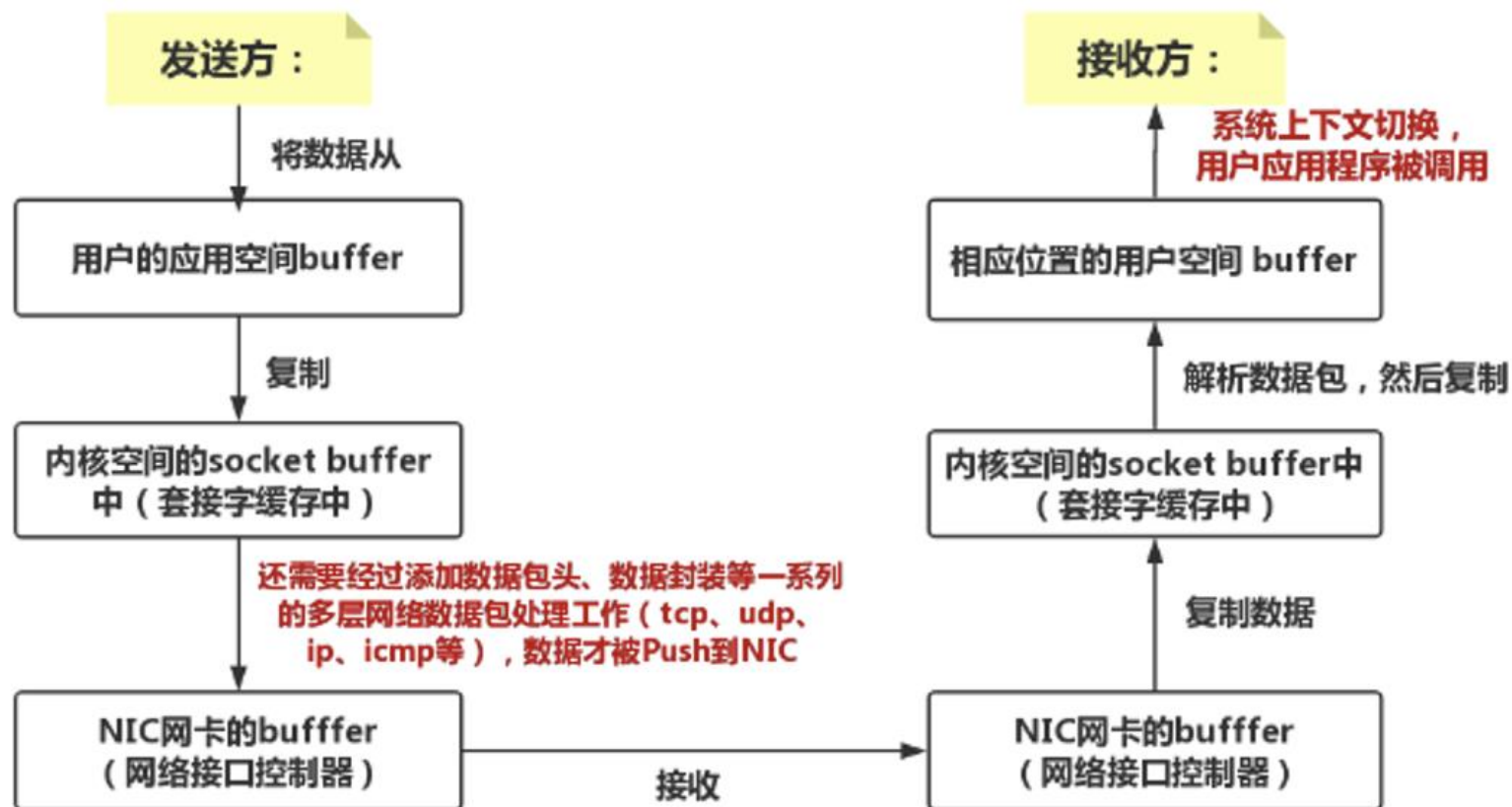
- 网络进程就是在网络结点计算机上运行的程序。
- 网络编程一般是指利用不同层次的通信协议提供的接口实现网络进程通信的编程。
 - C/S , B/S , $B/S+C/S$ 等
- 传输层的网络进程通信机制是进行网络编程的基础。
 - 基于Socket的编程

Socket（套接字）

- **Socket**提供**给应用程序编程的接口和一种机制**；
- 为给定的IP和Port，在应用程序中创建**Socket**，通过一种绑定机制与驱动程序建立关系
- 可以把Socket比喻成一个港口码头，应用程序只要把货物放到港口码头上，就算完成了货物的运送。应用程序只需等待货物到达码头后，将货物取走

Socket数据发送过程

传统网络通信模式的流程：



Java中网络编程类

- Java中网络编程类位于 **java.net** 包中。
- 很多应用程序需要可靠的、按顺序的数据传输，也有的应用程序不需要，因此在设计网络程序时应正确选择网络类。
 - **URL、URLConnection、Socket和ServerSocket** 类是 **基于TCP协议** 的；
 - Socket类用于TCP通信的服务器和客户端。
 - ServerSocket类用于TCP通信的服务器端。
 - **DatagramPacket、DatagramSocket和MulticastSocket** 类是 **基于UDP协议** 的。
 - DatagramSocket类用于UDP通信。

主要内容

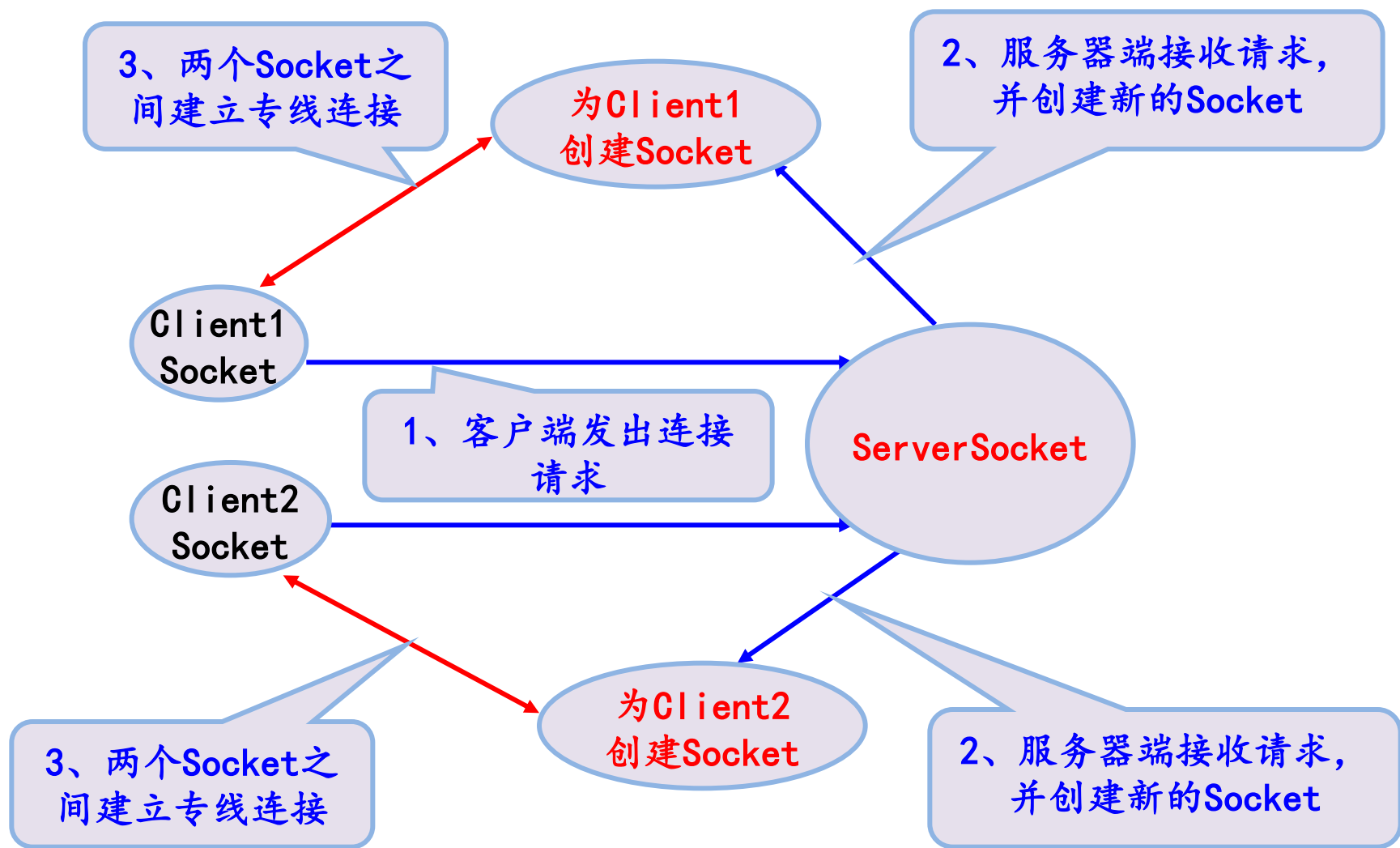
- 网络编程的基础知识
- **基于TCP的网络程序设计**
- 基于UDP的网络程序设计
- 访问Internet网络资源

- 利用TCP协议进行通信的两个应用程序是有主从之分的，一个称为服务器程序，一个称为客户机程序，两者的功能和编写方法不大一样。

- TCP服务器程序类似114查号台，而TCP客户机程序类似普通电话。必须先有114查号台。普通电话才能拨打114，在114查号台那边是先有一个总机，总机专门用来接听拨打进来的电话，并不与外面的电话直接对话，而是将接进来的电话分配到一个空闲的座机上，然后由这个座机与外面的电话直接对话。
- 总机在没有空闲的座机时，可以让对方排队等候，但等候服务的电话达到一定数量时，总机就会彻底拒绝以后再拨打进来的电话。

- Java中提供的ServerSocket类用于完成类似114查号台总机的功能,
- Socket类用于完成普通电话和114查号台端的座机功能。

TCP网络程序的工作原理



TCP客户端程序与TCP服务器端程序的交互过程

- 1) 服务器端创建一个ServerSocket，然后调用accept方法等待客户来连接；
- 2) 客户端程序创建一个Socket并请求与服务
- 器建立连接；
- 3) 服务器端接收客户的连接请求，并创建一个新的Socket与该客户建立专线连接；
- 4) 建立了连接的两个Socket在一个单独的线程(由服务器程序创建)上对话；
- 5) 服务器开始等待新的连接请求，当新的连接请求到达时，重复步骤2-5的过程。

InetAddress类

- InetAddress是用于表示计算机IP地址的一个类，而在日常应用中的计算机地址是用“192.168.0.1”、www.sina.com.cn等字符串格式来表示的。
- **getName(String host) 方法：**
 - 通过**域名**来构造类InetAddress的实例对象
- **getByAddress(byte[] addr) 方法**
 - 通过**4个字节的网络地址**构造类InetAddress的实例对象

InetDemo.java

```
import java.net.*;
```

```
public class InetDemo {
```

Run | Debug

```
    public static void main(String[] args) {
```

```
        InetAddress ip = null;
```

```
        try {
```

```
            ip = InetAddress.getByName(host: "www.baidu.com");
```

```
        } catch (Exception e) {
```

```
            System.out.println(e);
```

```
        }
```

```
        if(ip != null){
```

```
            System.out.println("Host Name: " + ip.getHostName());
```

```
            System.out.println("IP Address: " + ip.getHostAddress());
```

```
        }
```

```
        else
```

```
            System.out.println(x: "Cannot access www.baidu.com!");
```

```
    }
```

```
}
```

Host Name: www.baidu.com
IP Address: 39.156.66.14

简单的Client/Server程序

- 客户端和服务端采用TCP协议进行通信，当客户端链接服务器端程序后，客户端程序向服务器端程序发送用户从控制台输入的内容。

简单的Client/Server程序---Server端

```
public static void main(String[] args) {
```

```
    try (ServerSocket serv = new ServerSocket(port: 9090)) {
```

1. 创建serversocket,
并与端口绑定

```
        System.out.printf(format: "Bind Port %d\n", serv.getLocalPort());
```

```
        Socket socket = null;
```

```
        while (true) {
```

```
            // 接收连接, 如果没有连接, accept() 方法会阻塞
```

2. 等待客户端
连接

```
            socket = serv.accept();
```

```
            System.out.println("Connected with " + socket.getInetAddress());
```

3. 连接成功,
并打印客户
端信息

```
            // 获取输入流, 并使用 BufferedInputStream 和 InputStreamReader 装饰, 方  
            便以字符流的形式处理, 方便一行行读取内容
```

```
            try {
```

```
                BufferedReader in = new BufferedReader(new InputStreamReader  
                (socket.getInputStream()));
```

4. 获取客户端输入或输出流

```
                socket.getInputStream().read();
```

```
                String msg = null;
```

```
                char[] cbuf = new char[1024];
```

```
                int len = 0;
```

```
                while ((len = in.read(cbuf, off: 0, len: 1024)) != -1) { // 循  
                环读取输入流中的内容
```

```
                    msg = new String(cbuf, offset: 0, len);
```

```
                    System.out.printf(format: "Received Message --> %s \n",  
                    msg);
```

```
                    if ("Bye".equals(msg))
```

```
                        break;
```

5. 与客户端
交互

```
            }
```

```
            socket.close();
```

5. 关闭socket

```
        } catch (IOException e) {
```



简单的Client/Server程序---Client端

```
public static void main(String[] args){  
    try(Socket socket = new Socket(host: "localhost", port: 9090)){  
        BufferedWriter out = new BufferedWriter(new OutputStreamWriter(socket.  
            getOutputStream()));  
        Scanner scanner = new Scanner(System.in);  
        String msg = null;  
        while( true ){  
            msg = scanner.next();  
            System.out.printf(format: "Send Msg --> %s \n", msg);  
            out.write(msg);  
            out.flush(); // 立即发送, 否则需要积累到一定大小才一次性发送  
            if(msg.equals(anObject: "Bye"))  
                break;  
        }  
        socket.close();  
    }catch (IOException e){  
        e.printStackTrace();  
    }  
}
```

1. 创建socket, 注意ip地址和端口号

2. 获取客户端输入或输出流

3. 与服务器端交互

4. 关闭socket

简单的Client/Server程序

客户端输出：

```

Lesson_10_Func
Hello
Send Msg --> Hello
World
Send Msg --> World
Bye
Send Msg --> Bye

```

服务器端输出：

```
Bind Port 9090
Connected with /127.0.0.1
Received Message --> Hello
Received Message --> World
Received Message --> Bye
Disconnected with /127.0.0.1
```

ServerSocket的常用方法

ServerSocket();	创建一个未绑定的ServerSocket
-----------------	----------------------

ServerSocket(int port);	绑定到指定的port上,连接队列默认50
-------------------------	----------------------

ServerSocket(int port, int backlog);	指定最大连接队列
--------------------------------------	----------

ServerSocket(int port, int backlog, InetAddress bindAddr);	
--	--



Socket accept();	void bind(SocketAddress endpoint);
------------------	------------------------------------

void bind(SocketAddress endpoint, int backlog);	
---	--

SocketAddress getLocalSocketAddress();	void close();
--	---------------

InetAddress getInetAddress();	int getLocalPort();
-------------------------------	---------------------

boolean isBound();	boolean isClosed();
--------------------	---------------------

void setSoTimeout(int timeout);	int getSoTimeout();
---------------------------------	---------------------

ServerSocket

- 构造函数
 - `public ServerSocket () ;`
 - `public ServerSocket (int port) ;`
 - `public ServerSocket (int port, int backlog) ;`
 - `public ServerSocket (int port, int backlog, InetAddress bindAddr) ;`
- `close ()` 方法;
- `accept ()` 方法;

Socket的常用构造方法

Socket的常用构造方法

Socket();	创建一个未连接的Socket
------------------	-----------------------

Socket(InetAddress addr, int port);	指定IP地址和port
--	--------------------

Socket(InetAddress addr, int port, InetAddress localaddr, int localPort);
--

Socket(String host, int port, InetAddress localaddr, int localPort);

Socket(String host, int port);	Socket(SocketImpl impl);
---------------------------------------	---------------------------------

Socket的常用方法(续)

void bind(SocketAddress bindpoint); void close();

void connect (SocketAddress endpoint); int getPort();

void connect (SocketAddress endpoint, int timeout);

InetAddress getInetAddress(); int getLocalPort();

InetAddress getLocalAddress(); boolean isConnected();

SocketAddress getLocalSocketAddress();

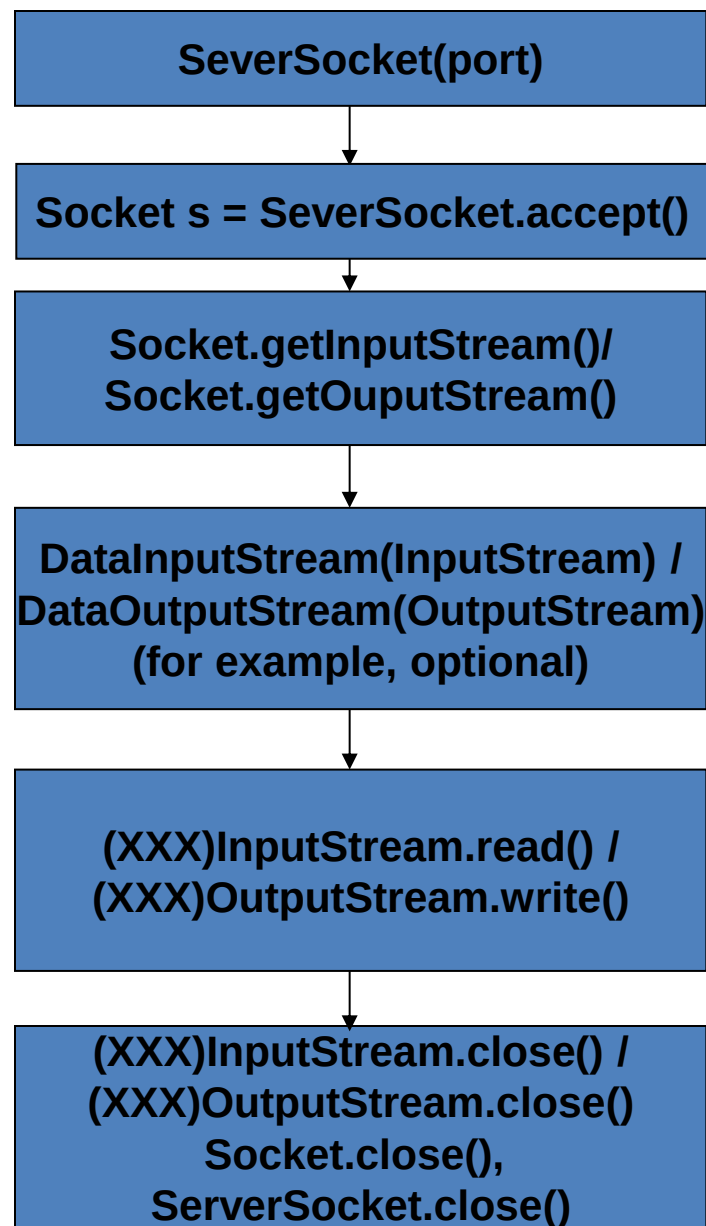
InputStream getInputStream(); boolean isClosed();

OutputStream getOutputStream(); boolean isBound();

SocketAddress getRemoteSocketAddress();

Typical Steps to Write a Socket Server

1. Create a server socket
2. Accept a connection request from a client and return a new socket
3. Get input stream / output stream from the accepted socket
4. Encapsulate raw input stream / output stream to high level streams according to data type (optional)
5. Receive/send message from/to streams
6. Release resources



简单的一对一聊天程序

- 目前程序只能单向发送消息
- 程序只能一对一聊天
- 因为程序是单线程的



```
public static void chatWithServer(Socket socket){
    try {
        BufferedReader in = new BufferedReader(new InputStreamReader(socket.
            getInputStream()));
        BufferedWriter out = new BufferedWriter(new OutputStreamWriter(socket.
            getOutputStream()));

        Scanner scanner = new Scanner(System.in);

        String msg = null;
        char[] cbuf = new char[1024];
        int len = 0;
        while (true) { // 循环读取输入流中的内容
            msg = scanner.nextLine();
            out.write(msg);
            out.flush(); // 立即发送, 否则需要积累到一定大小才一次性发送

            if((len = in.read(cbuf, off: 0, len: 1024)) != -1){
                msg = new String(cbuf, offset: 0, len);
                System.out.printf(format: "Server says: %s \n", msg);
            }
        }
    } catch (IOException e) {
```

ChatWithClient:
先写后读

```

public static void chatWithClient(Socket socket){
    try {
        BufferedReader in = new BufferedReader(new InputStreamReader(socket.
            getInputStream()));
        BufferedWriter out = new BufferedWriter(new OutputStreamWriter(socket.
            getOutputStream()));

        Scanner scanner = new Scanner(System.in);

        String msg = null;
        char[] cbuf = new char[1024];
        int len = 0;
        while (true) { // 循环读取输入流中的内容
            if((len = in.read(cbuf, off: 0, len: 1024)) != -1){
                msg = new String(cbuf, offset: 0, len);
                System.out.printf(format: "对方说: %s \n", msg);
            }

            msg = scanner.nextLine();
            out.write(msg);
            out.flush(); // 立即发送, 否则需要积累到一定大小才一次性发送
        }
    } catch (IOException e) {

```

ChatWithServer:
先读后写

客户端输出：

```
—  
Hello  
Server says: Hi  
How are you  
Server says: I am fine  
Bye  
Server says: Bye
```

服务器端输出：

```
Bind Port 9090  
Connected with /127.0.0.1  
对方说： Hello  
Hi  
对方说： How are you  
I am fine  
对方说： Bye  
Bye  
—
```


自由聊天程序

- 为了保证一次可以说多句，可以使用多线程，一个线程用来说，一个线程用来听。
互不干扰！！！！

服务器主线程

```
Thread read = new Thread(new Runnable() {  
    public void run() {  
        ChatUtlis.readFromSocket(in);  
    }  
});  
Thread write = new Thread(new Runnable() {  
    public void run() {  
        ChatUtlis.writeToSocket(out);  
    }  
});  
read.start();  
write.start();  
  
read.join();  
write.join();  
  
socket.close();
```

读线程

```
public static void readFromSocket(BufferedReader in){  
    try {  
        String msg = null;  
        char[] cbuf = new char[1024];  
        int len = 0;  
        while ((len = in.read(cbuf, off: 0, len: 1024)) != -1) { // 循环读取输入  
            msg = new String(cbuf, offset: 0, len);  
            System.out.printf(format: "Other says: %s \n", msg);  
        }  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

写线程

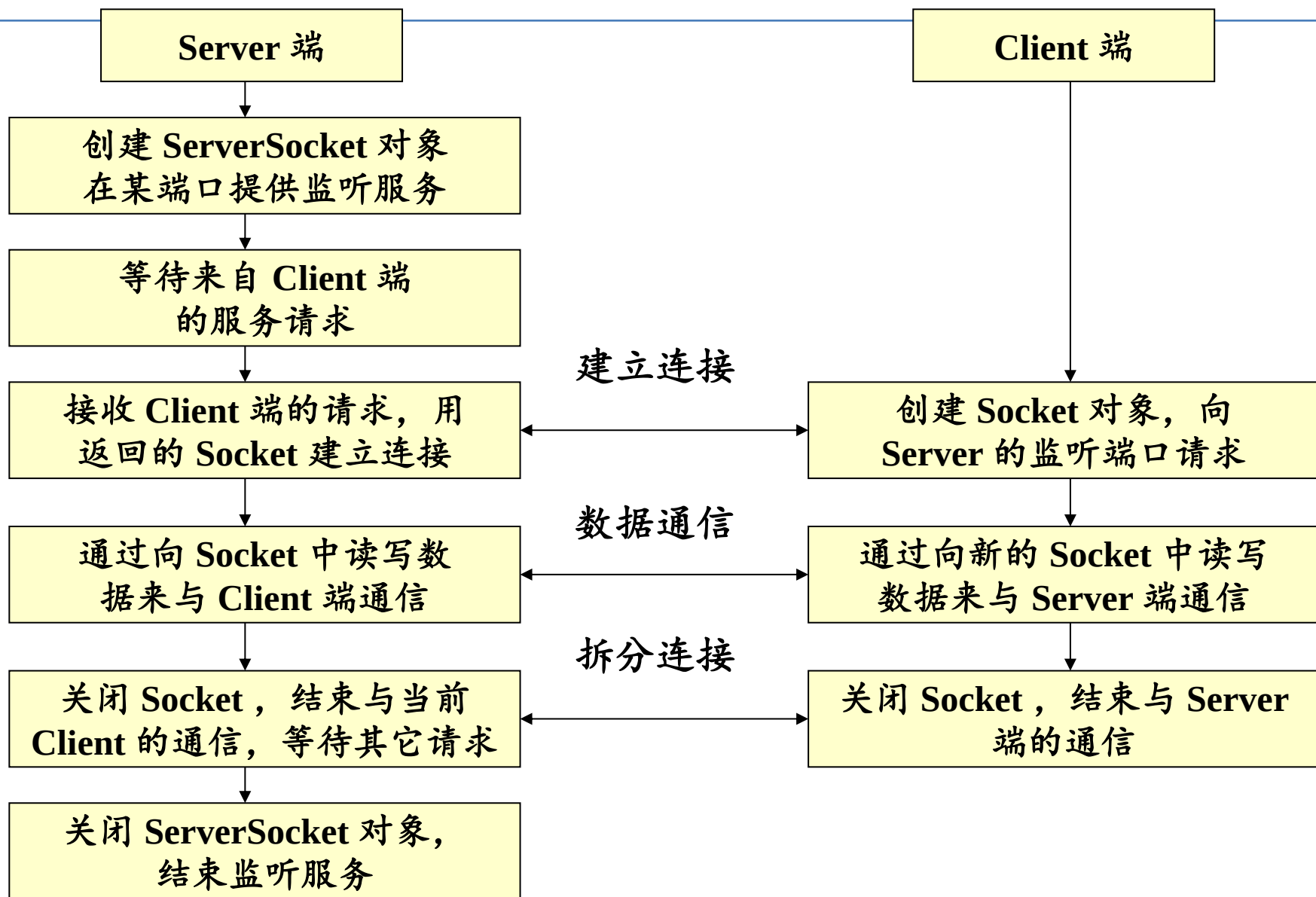
```
public static void writeToSocket(BufferedWriter out){  
    try {  
        Scanner scanner = new Scanner(System.in);  
  
        String msg = null;  
        while( true ){  
            msg = scanner.nextLine();  
            out.write(msg);  
            out.flush(); // 立即发送, 否则需要积累到一定大小才一次性发送  
        }  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

完善的TCP服务器程序模型（思考题）

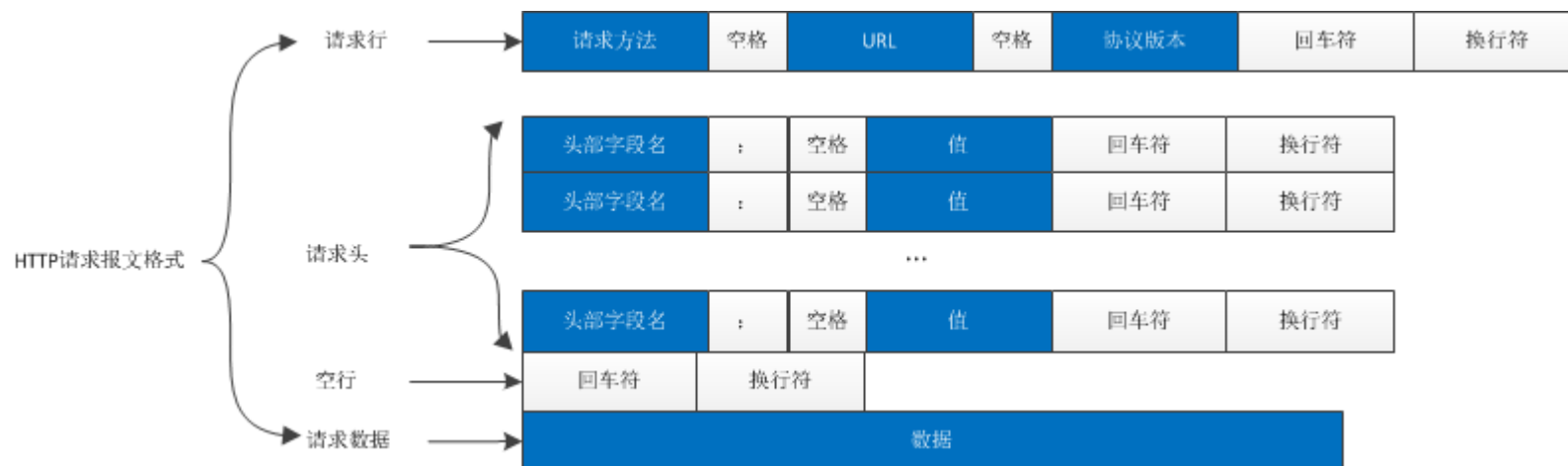
- 编程实例：
 - 服务器程序能够同时与多个客户端对话
- TCP服务器程序模型的编写要点：
 - TCP服务器程序要想能接收多个客户端连接，需要循环调用`ServerSocket.accept`方法；
 - 服务器程序与每个客户端连接的会话过程不能互相影响，需要在独立的线程中进行；
 - 一个线程服务对象与一个服务器端`Socket`对象相关联，共同来完成一个客户端的会话。

在TCP网络连接上传递对象

- `ObjectInputStream`和`ObjectOutputStream`可以用来包装底层网络字节流，TCP服务器和TCP客户端之间就可以传递对象类型的数据，实现从底层输入流中读取对象类型的数据和将对象类型的数据写入到底层输出流。



- 如何使用Socket实现HTTP协议



主要内容

- 网络编程的基础知识
- 基于TCP的网络程序设计
- **基于UDP的网络程序设计**
- 访问Internet网络资源



基于UDP的网络程序设计

- UDP网络程序的工作原理
- DatagramSocket 类
- DatagramPacket 类
- InetAddress 类
- 最简单的UDP程序
- 用UDP编写网络聊天程序

- 用户数据报协议UDP (user datagram protocol) 是一个无连接的、发送独立数据包的协议，它不保证数据按顺序传送和正确到达。
- 数据报Socket又称为UDP套接字，它无需建立、拆除连接，而是直接将信息打包传向指定的目的地，使用简单，占用资源少，适合于断续、非实时通信。
- 利用UDP通信的两个程序是平等的，没有主次之分，两个程序的代码可以完全一样。

DatagramSocket 类

- 构造函数：
 - `Public DatagramSocket () ;`
 - `Public DatagramSocket (int port) ;`
 - `Public DatagramSocket (int port, InetAddress laddr) ;`
- `Close ()` 方法;
- `Send (DatagramPacket p)` 方法
- `Receive (DatagramPacket p)` 方法

DatagramPacket 类

- 如果把DatagramSocket比作创建的港口码头，那么DatagramPacket就是发送和接收数据的集装箱。
- 构造函数
 - `Public DatagramPacket(byte[] buf, int length);`
 - `Public DatagramPacket(byte[] buf, int length, InetAddress address, int port);`
- `getInetAddress()` 和 `getPort()` 方法;
- `Byte[] getData()` 和 `getLength()` 方法;

最简单的UDP程序

- 字符串与字节数组之间的双向转换。
- UDP接收程序必须先启动运行，才能接收UDP发送程序发送的数据。
- 举例：
 - [UdpSend.java](#) [UdpRecv.java](#)

UDP sender

```
public class UdpSend {  
    public static void main(String[] args) throws Exception {  
        DatagramSocket ds = new DatagramSocket();  
        String str = "hello world";  
        DatagramPacket dp = new DatagramPacket(str.getBytes(), str.length(),  
            InetAddress.getByName("127.0.0.1"), 3000);  
        ds.send(dp);  
        ds.close();  
    }  
}
```

UDP Receiver

```
public class UdpRecv {  
    public static void main(String[] args) throws Exception {  
        DatagramSocket ds = new DatagramSocket(3000);  
        byte[] buf = new byte[1024];  
        DatagramPacket dp = new DatagramPacket(buf, 1024);  
        ds.receive(dp);  
        String strRecv = new String(dp.getData(), 0, dp.getLength()) + " from "  
            + dp.getAddress().getHostAddress() + ":" + dp.getPort();  
        System.out.println(strRecv);  
        ds.close();  
    }  
}
```


UDP网络程序的编写步骤

- 第一步：服务器端/客户端创建DatagramSocket实例对象；
- 第二步：编写send()和receive()方法
 - DatagramSocket.send(DatagramPacket p)；
 - DatagramSocket.receive(DatagramPacket p)；
- 第三步：DatagramSocket.close()；

- 理解 URL 类是对统一资源定位符的抽象
- 理解 IP、port、InetAddress 类
- 理解 Socket
- 掌握基于 TCP 的编程模式
- 掌握基于 UDP 的编程模式
- 基于 UDP 的通信和基于 TCP 的通信不同，基于 UDP 的信息传递更快，但不提供可靠性保证。

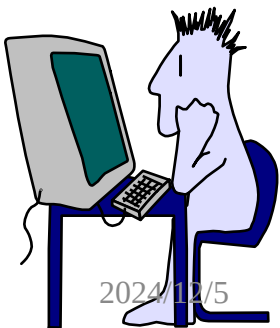
Questions?

End of Class

高祥: <https://gaoxiang9430.github.io>

邮箱: xiang_gao@buaa.edu.cn

办公室: 工训楼 (软件学院) 东603



2024/12/5

Xiang Gao



北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院